Project APEX

Chapter 25 — Event Bus & Messaging

25.1 Purpose

The Event Bus provides asynchronous communication between Project APEX microservices. It enables services to publish and consume events without tight coupling, improving scalability, resilience, and extensibility.

25.2 Objectives

• Event-driven architecture
• Loose service coupling
• Reliable message delivery
• Horizontal scalability
• Event replay and auditing

25.3 Core Components

• Event Producers
• Event Bus / Broker
• Topics & Queues
• Event Consumers
• Dead Letter Queue (DLQ)
• Schema Registry

25.4 Event Flow

Producer → Event Serialization → Event Bus → Topic/Queue → Consumer → Processing → Acknowledgement → Monitoring.

25.5 Message Types

• Domain Events
• Integration Events
• Commands
• Notifications
• System Events

25.6 Design Principles

• Asynchronous by default
• Idempotent consumers
• At-least-once delivery
• Versioned event schemas
• Observability and traceability

25.7 Challenges

Distributed messaging requires handling duplicate events, ordering, retries, poison messages, backpressure, and eventual consistency across services.

25.8 Role in APEX

The Event Bus connects observation, memory, planning, execution, monitoring, and feedback services into a cohesive event-driven platform capable of real-time workflow processing.

Component	Primary Responsibility
Producer	Publish domain events
Broker	Route and persist messages
Topic/Queue	Organize event streams
Consumer	Process incoming events
DLQ	Store failed messages
Schema Registry	Manage event contracts
Monitoring	Track message health

25.9 Chapter Summary

The Event Bus & Messaging layer enables scalable, resilient, and loosely coupled communication across Project APEX. It forms the backbone of the platform's event-driven architecture, ensuring reliable propagation of workflow events and system state changes.